The picture can't be displayed.

Low-Level Design Document (HLD)

RAG Customer Support Assistant

Prepared By
Sannidhya Milind Bahulekar

Internship ID
IN226066402

College / University
Ajeenkya D.Y. Patil University

Internship Organization
Innomatics Research Labs

Submission Date
24 April 2025

LOW-LEVEL DESIGN DOCUMENT

RAG-Based Customer Support Assistant

Internal Implementation Design with LangGraph, ChromaDB & HITL

1. Module-Level Design

This section defines the internal structure of each module, including its inputs, outputs, and internal logic. The system is composed of eight modules that operate in sequence. Each module has a single, well-defined responsibility.

This low-level design was developed after finalising the HLD during my internship at Innomatics Research Labs and reflects real implementation decisions made during hands-on testing

1.1 Document Processing Module

Purpose

Accepts a PDF file path as input and returns a list of raw page-level text strings. Handles errors such as encrypted PDFs, corrupted files, or empty pages.

Internal Logic

- Open the PDF using PyMuPDF (fitz.open)
- Iterate over each page and call page.get_text('text') to extract plain text
- Strip leading/trailing whitespace and skip pages with fewer than 20 characters (likely scanned images or blank pages)
- Return a list of (page_number, text) tuples for downstream processing

Key Data Structures

```
PageContent = { page_num: int, text: str, source_file: str }
```

1.2 Chunking Module

Purpose

Splits page-level text into overlapping chunks suitable for embedding. Produces a flat list of chunk objects with associated metadata.

Internal Logic

- Initialize LangChain's RecursiveCharacterTextSplitter with chunk_size=512, chunk_overlap=50
- Feed each page's text through the splitter
- Annotate each resulting chunk with its source file and page number
- Assign a unique chunk ID (UUID or sequential integer) to each chunk

Key Data Structures

```
Chunk = { chunk_id: str, text: str, source_file: str, page_num: int, token_count: int }
```

Configuration

chunk_size=512 and chunk_overlap=50 are chosen as defaults. These values balance context richness against embedding cost. They should be treated as tunable hyperparameters during evaluation.

1.3 Embedding Module

Purpose

Converts a list of text strings (chunks or query) into dense vector representations. Must use the same model for both ingestion and query embedding to ensure vector space compatibility.

Internal Logic

- Load the model once at startup (all-MiniLM-L6-v2 via SentenceTransformers, or OpenAI client)
- Accept a list of strings and batch-encode them to produce a list of float arrays
- Return embeddings in the same order as input strings

Key Data Structures

```
Embedding = List[float]  # length 384 (MiniLM) or 1536 (OpenAI)
```

1.4 Vector Storage Module

Purpose

Persists chunk embeddings and metadata to ChromaDB. Provides methods to insert, query, and delete records.

Internal Logic

- Initialize ChromaDB client with persist_directory='./chroma_db'
- Create or load a collection named 'support_docs'
- On ingestion: call collection.add(ids, embeddings, documents, metadatas)
- On query: call collection.query(query_embeddings, n_results=k) and return documents + distances

Schema

```
ChromaRecord = { id: str, embedding: List[float], document: str, metadata: { source: str,
page: int } }
```

1.5 Retrieval Module

Purpose

Given a query string, returns the top-k most semantically relevant chunks along with their similarity scores.

Internal Logic

- Embed the query string using the Embedding Module
- Query ChromaDB with n_results=k (default k=4)
- Convert raw L2 distances to a normalized similarity score: score = 1 / (1 + distance)
- Return a list of RetrievedChunk objects sorted by score descending

Key Data Structures

```
RetrievedChunk = { text: str, source: str, page: int, similarity_score: float }
```

1.6 Query Processing Module

Purpose

Composes the LLM prompt from the retrieved chunks and query, calls the LLM, and parses the response.

Prompt Template

```
System: You are a helpful customer support assistant.
Answer the user's question using ONLY the context provided below.
If the context does not contain enough information, say 'I am not able to
answer this from the available documentation.'

Context:
{retrieved_chunks_text}

User Question: {query}
```

Internal Logic

- Build prompt string by joining retrieved chunk texts with separator lines
- Call LLM API with the composed prompt
- Parse response text and extract confidence signal (via heuristic keyword detection or logprobs)
- Return answer string and confidence score

1.7 Graph Execution Module (LangGraph)

Purpose

Orchestrates the end-to-end query handling workflow using a stateful directed graph.

Graph State

```
class GraphState(TypedDict):
    query: str
    embedded_query: List[float]
    retrieved_chunks: List[RetrievedChunk]
    answer: str
    confidence: float
    escalated: bool
    human_response: Optional[str]
    session_id: str
    timestamp: str
```

Nodes

- input_node: validates and embeds the query
- process_node: retrieves chunks and generates answer
- output_node: formats and returns the final response
- hitl_node: creates ticket, suspends graph, waits for human input

Edges

- START -> input_node (unconditional)

- input_node -> process_node (unconditional)
- process_node -> router (conditional edge function)
- router -> output_node if confidence >= 0.75
- router -> hitl_node if confidence < 0.75 or intent == 'escalate'
- hitl_node -> output_node after human response received

1.8 HITL Module

Purpose

Manages the escalation path: ticket creation, suspension, human response collection, and state injection.

Internal Logic

- Generate a unique ticket ID (format: TKT-YYYYMMDD-XXXX)
- Write ticket to a local JSON file or database table with status='open'
- Call LangGraph's interrupt() to suspend the graph thread
- Return ticket ID to user immediately with message: 'Your query has been escalated. Reference: TKT-...'
- When agent submits response via CLI command or API endpoint, resume graph with human response in state
- Update ticket status to 'resolved' and log the full interaction

2. LangGraph Workflow Design

2.1 Node Definitions

Node Name	Input from State	Output to State
input_node	query	embedded_query
process_node	embedded_query	retrieved_chunks, answer, confidence
router	confidence, query	routes to next node (no state change)
output_node	answer OR human_response	Final response to user
hitl_node	query, retrieved_chunks	human_response, escalated=True

2.2 Conditional Routing Logic

The router function is a pure function that takes the current GraphState and returns the name of the next node as a string. The routing criteria are:

Condition	Next Node
confidence >= 0.75 AND no escalation keywords	output_node
confidence < 0.75	hitl_node

zero chunks retrieved	hitl_node
query contains: 'refund', 'cancel', 'legal', 'complaint'	hitl_node
LLM response contains 'I am not able to answer'	hitl_node

3. Complete Data Structure Reference

3.1 Query-Response Schema

```
QueryInput  = { query: str, session_id: str, timestamp: str }
QueryOutput = { answer: str, source_chunks: List[str], confidence: float,
                escalated: bool, ticket_id: Optional[str] }
```

3.2 Ticket Schema

```
Ticket = {
  ticket_id: str,          # TKT-YYYYMMDD-XXXX
  session_id: str,
  query: str,
  retrieved_chunks: List[str],
  llm_attempt: str,         # LLM answer before escalation
  status: 'open' | 'resolved',
  agent_id: Optional[str],
  human_response: Optional[str],
  created_at: str,
  resolved_at: Optional[str]
}
```

4. API / Interface Design

4.1 CLI Interface (Prototype)

The command-line interface supports three primary commands:

```
python main.py ask --query 'What is your return policy?'
python main.py ingest --file knowledge_base.pdf
python main.py resolve --ticket TKT-20250101-0042 --response 'The return window is 30 days.'
```

4.2 Interaction Flow

1. User calls: python main.py ask --query '...'
2. System embeds query, retrieves chunks, generates answer, evaluates confidence
3. If high confidence: prints answer with source citations
4. If low confidence: prints 'Escalated. Reference: TKT-...' and suspends
5. Agent calls resolve command to inject human response

6. System resumes, delivers response, logs interaction

5. Error Handling

Error Condition	Detection Method	Handling Strategy
PDF load failure	Exception catch in loader	Return error message; do not crash pipeline
Empty chunk list	len(chunks) == 0 after split	Log warning; skip document
Embedding API timeout	requests.exceptions.Timeout	Retry up to 3 times with backoff
Zero retrieval results	len(results) == 0 from ChromaDB	Set confidence=0.0; route to HITL
LLM API failure	HTTP 5xx or timeout	Retry once; if failure persists, escalate to HITL
Invalid query input	Empty string or len < 3	Return validation error to user immediately

6. Summary

This low-level design document provides the internal implementation specification for every module in the RAG Customer Support Assistant. Each module is self-contained with clear input/output contracts, making the system easy to test in isolation. The LangGraph workflow provides a transparent control structure where routing decisions are explicit and traceable. The HITL module ensures that no query falls through the cracks — when the automated system cannot answer confidently, a human picks it up without disrupting the user experience.